

Automatiserad testning av backend-API i Node

Undersökning och implementering av en teststrategi

Olof Zetterlund

2026-05-25

Förord

Tack till Helena Johansson på Klarr AB för all feedback och hjälp och till Adam Falk som utvecklat teststrategin tillsammans med mig.

Abstract

Developing applications is usually done in teams that may change over time and developers often work with different components that rely on each other. Not having automated testing in this environment risks critical bugs and security issues going unnoticed. This work develops and implements a strategy for testing in a backend API developed in Node by examining widely used frameworks and tools. The implementation focuses on ease of use and maintainability with integration testing over thoroughness. The result is a stack of tooling based on Mocha, Supertest and Chai for creating and maintaining tests, Sinon for creating test doubles and Testcontainers to run an isolated database for testing. The work also includes documentation to help developers understand and apply the strategy.

Lista över förkortningar och förteckningar

API	Application programming interface	API:er fungerar som broar mellan applikationer, vilket låter dem kommunicera och dela data
	Assert	En jämförelse mellan verkligt och förväntat resultat från ett test för att ange om det är godkänt eller underkänt
	Boilerplate-kod	Återkommande standardkod
	Docker	Öppen plattform som tillåter applikationer att köras i isolerade containers
	Endpoint	Kommunikationspunkt ett API exponerar över HTTP
	Flaky test	Test som ibland är godkänt och ibland inte på ett icke deterministisk vis
HTTP	Hypertext transfer protocol	Kommunikationsprotokoll som överför information mellan klient och server
	Seed	Lägga in test eller exempeldata i en databas
	Pull request	Begäran att lägga till eller uppdatera kod i ett projekt

Innehållsförteckning

Förord..... 2
Abstract..... 3
Lista över förkortningar och förteckningar..... 4
Innehållsförteckning..... 5
1 Inledning..... 6
1.1 Bakgrund..... 6
1.2 Syfte och mål..... 6
1.3 Avgränsningar..... 6
2 Kunskapsbas / Teoretisk grund..... 7
3 Metod..... 8
3.1 Val av arbetsprocess..... 8
3.2 Datainsamlingsmetoder..... 8
3.3 Analysmetoder..... 8
3.4 Etiska överväganden..... 8
4 Genomförande..... 9
5 Resultat..... 10
6 Slutsatser..... 11
6.1 Rekommendationer..... 11
7 Referenser..... 12
7.1 Hänvisning till böcker..... 12
7.2 Hänvisning till internet/webbsida..... 12
8 Bilagor..... 13

1 Inledning

1.1 Bakgrund

I moderna mjukvaruprojekt arbetar ofta flera utvecklare parallellt med olika delar av ett system. När utvecklare tillkommer och lämnar projektet kan det uppstå bristande förståelse för helheten, vilket ökar risken för att förändringar orsakar oväntade fel i andra delar av systemet.

I det studerade projektet saknas automatiserad testning, vilket har lett till att ändringar i kodbasen vid flera tillfällen orsakat kritiska buggar som hade inneburit säkerhetsproblem för en driftsatt applikation.

För att minska dessa risker är det viktigt att använda effektiva teststrategier som kan säkerställa att systemet fungerar korrekt även efter förändringar. Detta arbete fokuserar därför på att undersöka och implementera lämpliga teststrategier i en befintlig kodbas vilket gör det relevant ur ett arbetslivsperspektiv då effektiv automatisk testning är viktig i alla mjukvaruprojekt.

1.2 Syfte och mål

Syftet med detta examensarbete är att undersöka möjliga teststrategier och implementera ett ramverk för testning som är lämpligt för att säkerställa stabilitet i ett mjukvaruprojekt där flera utvecklare arbetar parallellt och där beroenden mellan olika delar av systemet är kritiska. Ramverket ska vara utformat så att nya systemutvecklare ska kunna förstå och använda sig av det för att säkerställa kvalitet och säkerhet i projektet.

Målet är att identifiera relevanta teststrategier, implementera dessa i en befintlig kodbas samt utvärdera hur de påverkar systemets stabilitet och utvecklingsprocess. Arbetet ska också ta fram en struktur och dokumentation för hur teststrategier kan upprätthållas över tid och förankra detta hos projektets medarbetare.

1.3 Avgränsningar

Detta arbete avgränsas till att undersöka och implementera teststrategier i ett specifikt mjukvaruprojekt som varit under utveckling i över ett år där en stor del av utvecklingen utförs av LIA-praktikanter. Eftersom applikationen fortfarande är under utveckling och testningen ska innebära så lite extra arbete som möjligt för utvecklarna måste fokus ligga enbart på de mest kritiska delarna. I detta fall omfattar det affärslogiken i API:et, datavalidering av indata och transaktionsprecision i databaslagring medan frontend och yttre tjänster inte ingår i arbetet.
Implementering ska vara enkel att underhålla samtidigt som den ska täcka så stor del av kodbasen som möjligt. Arbetet omfattar inte jämförelse av alla existerande teststrategier, utan fokuserar på de strategier som bedöms vara mest relevanta för projektets behov och önskemål. Vidare begränsas arbetet till den aktuella kodbasen och dess utvecklingsmiljö, vilket innebär att resultatet inte är direkt generaliserbart till andra typer av projekt.

2 Kunskapsbas / Teoretisk grund

2.1 Teststrategier

En fullständig teststrategi för en internetapplikation innebär tester av allt från användar-gränssnitt för webbsidan och svarstider till nätverksredundans och kapacitet (Myers et al., 2012).
Modern mjukvarutestning delas ofta in i tre delar. Enhetstestning, integrationstestning och end-to-end-testning. Enhetstester är snabba och precisa men fokuserar på de minsta delarna medan end-to-end-tester garanterar kvalitet men tar lång tid att köra och är ofta tidskrävande att skriva. Mitt emellan hamnar integrationstester som ger en bra kvalitetssäkring och är snabbare att utveckla då ett test kan täcka en stor del av kodbasen samtidigt som de går snabbt att skriva (daCosta, 2021).

2.2 Continous Integration

Continous Integration (CI) innebär att kod kontinuerligt integreras i en gemensam kodbas. Det ställer krav på fungerande och välstrukturerad kod och för att uppnå detta används

automatiska kontroller som kan bygga projektet, validera kodstruktur och köra en testsvit (daCosta, 2021). GitHub är en molnplattform för versionshantering och samarbete vid kodutveckling som hanterar CI med workflow-filer i GitHub Actions. GitHub Actions kan automatisera allt saker som byggande av kod, säkerhetskontroller och testkörning (GitHub, 2026).

2.3 Testramverk

Testramverk är verktyg som hjälper till vid utveckling av tester genom att organisera tester i ett läsbart format och visa upp resultatet på ett meningsfullt sätt (daCosta, 65). Inom Node finns det tre vanligt förekommande ramverk: Mocha, Vitest och Jest. Jest utvecklades från början av Facebook men övergick 2022 till OpenJS foundation (Jest, 2026). Jest fokuserar på enkelhet vilket innebär att det innehåller mycket funktionalitet från grunden och är enkelt att implementera. Då det är utvecklat för React rekommenderas det för React-frontend men används ofta även i Node backend. Mocha är populärt för testning i ett backend-API med Node då det bara innehåller det mest nödvändiga samtidigt som det kan anpassas genom externa kodbibliotek (Kan, 2026). Vitest är speciellt utvecklat för projekt som kör Vite i frontend men precis som Jest fungerar det utmärkt att använda i backend. Dessa testramverk använder så kallade hooks för att initiera och städa upp efter testsviten. Innan tester initieras körs kod i en before eller beforeEach-hook och efter testerna körs after och afterEach-hookar (Mocha, 2026).

2.4 Assertion libraries

Assertion libraries används för att skapa och utföra jämförelser för att godkänna eller underkänna tester. De är inkluderade i testramverk som Jest och Vitest men kan även vara fristående som Chai (Wagner, 2025), som har stöd för olika sätt att skriva tester på beroende på vad utvecklarna är vana vid sedan tidigare och tillåter en så kallade språkkedja av vanliga engelska ord som “to, be, is, that, which” och så vidare för öka läsbarheten i tester (Chai, 2026)

2.5 Test double

Inom testning behöver delar av mjukvaran ofta isoleras och brytas ut. Enhetstester ska fokusera på en liten enhet och inte påverkas av andra funktioner och vid integrationstester

behöver externa tjänster brytas ut för att inte påverka testerna. För att åstadkomma detta används så kallad mocking och stubbing (daCosta, 2021). Dessa begrepp används ofta synonymt men medan en stub returnerar hårdkodad data, ofta utan att använda någon logik, används en mock för mer kontroll och övervakning (Rasmusson, 2017).
Jest och Vitest har integrerad funktionalitet för mockning medan Mocha kräver ett externt bibliotek vilket ger flexibilitet för vilken typ av test doubles man vill använda. Ett komplement till Mocha är Sinon som tillhandahåller hjälpmedel till testramverk för att minska boilerplate-kod (Subramaniam, 2016).

2.6 Rapporteringsverktyg

Code coverage eller täckningsgrad visar hur stor del av applikationens kod som berörs av testkörning. Genom att mäta vilka delar av koden som täcks eller framförallt inte täcks kan man säkerställa att all funktionalitet och logik körs igenom vid automatisk testning (daCosta, 2021). Poängen med täckningsgrad är inte att uppnå full täckning utan att ha bra koll på vilka delar som inte testas (Fowler, 2012). Det finns många verktyg för att skapa rapporter på täckningsgrad men ett som använder Node.js inbyggda V8 engine direkt är C8 (Coe, 2018) Mochawesome används ofta tillsammans med Mocha för att visualisera en testsvit genom att skapa en HTML och CSS baserad rapport (Mochawesome, 2022).

2.6 Syntes

Att kvalitetssäkra utvecklingen i ett projekt ställer krav på en komplett strategi med automatiserade tester i en CI-pipeline och återkoppling till utvecklare. Ett testramverk tillsammans med ett assertion library räcker för att skriva och köra tester men för att isolera externa tjänster krävs även verktyg för att skapa test doubles. Baserat på den teoretiska grunden och krav på enkelt underhåll och fokus på de mest kritiska delarna så utvecklades en strategi med integrationstestning med Mocha, Chai och Supertest tillsammans med rapporteringsverktygen C8 och Mochawesome.

3 Metod

3.1 Val av arbetsprocess

Problemlösningsdriven utveckling har använts för att undersöka vilka strategier som är standard inom branschen och utifrån dessa ta fram ett ramverk som passar projektet. Jag har även observerat hur testning införts i andra stora publika projekt för att dra lärdom av dessa. Detta följdes av en implementeringsprocess där projektets behov först kartlades genom en genomgående analys av existerande kod för att förstå vad som kräver tester. För att förstå vilka metoder som är viktigast och möjliga att införa har förslag löpande diskuterats med projektledare.
Efter detta började en iterativ utvecklingsprocess där tester konstruerades och utvärderades och ett första utkast på strategi presenterades för övriga utvecklare som gav återkoppling som kunde användas för att förbättra strategin. Detta resulterade i en uppdaterad testdataseedning vilket krävde uppdatering av existerande tester och utveckling av hjälpfunktioner för att hantera seedning. Slutligen varvades implementeringsprocess med iterativ utveckling för att nå fram till den färdiga lösningen.

3.2 Datainsamlingsmetoder

Feedback har löpande samlats in från projektledare om strategival för att på bästa sätt tillgodose att lösningen vi utvecklade var väl anpassad efter de krav som ställdes.

Utvärdering av utformad strategi utfördes i en workshop där implementeringen kort förklarades för utvecklare i projektet för att de sedan, med hjälp av sammanställd dokumentation, fick pröva på att skriva tester och framföra sina åsikter. I workshopen deltog nio personer som svarade på frågor om vad de ansåg om tester och vad ett nytt arbetssätt med tester skapade för oro. Feedback samlades också in när lösningen var implementerad för att identifiera problem med dokumentation och möjliga förbättringar vid konstruerande av tester.

För att komma fram till rätt strategi för databashantering under lokal testning jämfördes tidsåtgången mellan att starta upp en lokal docker container genom testcontainers med att koppla upp sig mot en produktionsdatabas i molnet.

3.3 Analysmetoder

En stor del av insamlad data var feedback från projektledare som tillsammans med jämförelse av popularitet och branschstandarder stod till grund för valen av ramverk och hjälpbibliotek som implementerades för att uppnå målet med en teststategi som var relevant för aktuellt projekt.

Den kvantitativa datan från prestandamätningar av databashantering jämförde totaltid och uppmättes av Mocha för att utvärdera vilket val som var bäst lämpat för automatiska tester.

3.4 Etiska överväganden och AI-användning

All feedback som samlats in från utvecklare är generell och kan inte kopplas till en specifik person.
Eftersom API:ets kod är inte publik har ingen kod därifrån delats med utomstående eller AI. AI i form av Copilot har löpande använts under arbetets gång för att söka information som sedan har kontrollerats mot andra källor. AI har också använts vid felsökning av problem där lösningar presenterade av AI testats för att senare verifieras av officiell dokumentation. Kod som delats med AI har varit generell och endast kopplad till testutveckling.
Kodkomplettering i Visual Studio Code har använts vid koduppprepning i konstruktion av tester.

4 Genomförande

Projektet består av ett backend-API som varit under utveckling i över ett år, främst utvecklat med Node.js, Express och Mongoose och en MongoDB-dokumentdatabas. Arbetet startade med en analys av existerande kodbas och en riskanalys av funktionaliteten hos API:et för att komma fram till en relevant strategi för projektet. Detta innebar att gå igenom alla funktioner och databasmodeller för att identifiera vilka delar som var mest kritiska att testa.

Detta följdes av en utvärdering av existerande testramverk och strategier för att välja en lösning som fungerar för utvecklare utan tidigare erfarenhet av testning. Utvärderingen resulterade i tre likvärdiga alternativ för ramverk: Jest, Mocha och Vitest. Då projektet är ett API utvecklat i JavaScript med Node.js och Express.js föll valet på Mocha på grund av låga

systemkrav och stora möjligheter till anpassning. Då kraven på testerna var att de skulle vara heltäckande och enkla att underhålla föll det naturliga valet på integrationstestning vilket innebär att hela funktionaliteten testas från endpoint till databas till skillnad från enhetstester som fokuserar på att de minsta enheterna i en funktion fungerar. Det finns många automatiserade lösningar för integrationstester där molnbaserade tjänster valdes bort av ekonomiska skäl. I slutändan föll valet på Supertest som är vanligt i kombination med Mocha för integrationstestning av ett server-API då verktyget startar hela applikationen och kan hantera HTTP-begäran/svar och inloggade användare genom en agent.
När grunden för teststrategin var satt började ett iterativt arbete med att utveckla tester för att utvärdera de val som gjorts för hjälpverktyg. Mocha innebar ingen större skillnad från tidigare erfarenhet av Jest och Vitest och Supertest var enkelt att sätta sig in i och använda.

Figur 1

Exempel på hur supertest agent används vid konstruktion av test

```
await agent.post("/v1/admin/create-accounts")
      .expect(403)
      .expect({
        error: "Admin permission required!",
      });
```

Agenten anropar endpointen och förväntar sig en specifik statuskod och felmeddelande som svar

Då projektet använder externa tjänster för att hantera e-postutskick som inte ska hanteras av testsviten behövs en strategi för att exkludera dessa från testerna. Eftersom Mocha hade valts som testramverk som inte innehåller verktyg för att hantera test doubles, krävdes ytterligare utvärdering av alternativ som används i branchen och Sinon framstod som det mest utbredda. Det har en stark koppling till Mocha och implementerades i projektet för att isolera e-postklienten i projektet. Följande rad visar hur Sinon stubbar skicka-funktionen för att returnera det förväntade svaret.

```
sinon.stub(SESClient.prototype, "send").resolves();
```

Det är viktigt att tester fungerar helt självständigt och för att säkerställa detta skapas ny data inför varje test för att sedan tas bort direkt efter genom Mochas beforeEach och afterEach. Projektet inkluderade redan en modul för databas-seeding som till en början duplicerades och modifierades för att enkelt hantera testdata. För att minska kodduplicering beslutades det, efter diskussion med projektledare, om att seeding-modulen skulle uppdateras av ansvariga utvecklare för att hantera testdata genom en seed-data-konfiguration. Namngivning av tester är viktigt för att ge utvecklare tydlig information framförallt vid underkända tester (Engmark, 2020) vilket åstadkoms genom att använda “should” “when” (se Bilaga 1).
Tester utvecklades för en liten del av projektet tillsammans med dokumentation med information om teststrategin och hur tester skulle utvecklas. Detta visades upp i en workshop med utvecklare och projektledare där utvecklarna fick testa att skapa tester och ge feedback på implementation och dokumentation. Då responsen var att det var svårt att förstå hur testerna skulle konstrueras byggdes dokumentationen på för att förtydliga oklarheter som skillnaden mellan Chai och Supertest och i vilken ordning hookar som before och beforeEach kördes i.
Ett workflow för GitHub skapades för att automatiskt köra testsviten när ny kod implementerades och script som läste igenom testrapporter för att summera dessa.
Slutligen fick utvecklare skapa tester på egen hand för endpoints där det saknades som granskades i grupp innan implementering i kodbasen.

5 Resultat

Resultatet av arbetet är en implementerad teststrategi i ett backend-API utvecklat i Node och Express. Strategin innefattar integrationstestning av hela flödet i applikation från endpoint till databas, automatisk testkörning genom Github Actions och dokumentation för det nya arbetssättet och hur tester designas.

Den färdiga lösningen består av testramverket Mocha tillsammans med Supertest för HTTP-testning och Chai som assertion library. Sinon används för att isolera externa tjänster som e-postutskick från testning. En isolerad testdatabas skapas i en docker-container genom Testcontainers.

Tabell 1

Jämförelse av tidsåtgång vid olika databashantering för testning

Databas	Tidsåtgång (192 tester)	Fördelar	Nackdelar
Testcontainers	12s	Isolerad, snabbare databasanrop	Kräver Mongo Container Image, Långsam uppstart
MongoDb Atlas	22s	Används redan av API, Ingen installation	Långsam, krav på uppkoppling till Atlas

I en separat utvecklingsbranch körs testsviten automatiskt genom Github Actions vid varje pull request och alla tester måste vara godkända för att kod ska integreras. Information om underkända tester skrivs ut i Github Actions summering för att ge utvecklare direkt feedback vid en pull request. Detta är dock inte implementerat i huvudbranchen då det saknades tid för att utvärdera vilka effekter detta skulle få på arbetsflödet då det ökar tidsåtgången för en pull request.
HTML-rapporter av testresultat och täckningsgrad skapas av hjälpbiblioteken Mochawesome och C8 och laddas upp till Github efter automatisk testkörning. Dokumentation som beskriver hur implementerad teststrategi fungerar och hur tester konstrueras finns tillgänglig för utvecklare (se Bilaga 1).
Tester har konstruerats för en stor del av existerande kodbas och utvecklare har utbildats i det nya flödet och testat att själva utveckla tester. Detta resulterade i totalt 226 tester för 21 endpoints och en täckningsgrad av relevant kod på ca 68% av satser och 82% av grenar.

Användarfeedback visade att utvecklarna hade svårigheter med att skapa konfigurationer för testdata som skulle seedas inför test. Konstruerandet av testerna fungerade i övrigt väl med stöd av dokumentation och redan existerande tester som kunde användas som mall.

6 Slutsatser

Att implementera teststrategier i ett projekt som helt saknar automatiserade tester är inte i sig en lösning på problem som bristande säkerhet och kodkvalitet. Testning är ett ständigt pågående arbete som ställer krav på kunskap hos de som ska använda sig av strategin.
Då det inte har funnits tid för att undersöka hur teststrategin efterlevs i projektet går det inte att svara på hur väl det förankrats bland medarbetare. Det går inte heller att slå fast hur väl implementering står sig vid större kodförändringar vilket även skulle innebära mycket arbete för att uppdatera tester. Många medarbetare förstår vikten av effektiv testning men saknar intresset för att utveckla tester vilket innebär en risk att testning halkar efter.
Valet av integrationstestning möjliggör snabb utveckling av nya och uppdatering av existerande tester då alla tester följer samma format och hög täckningsgrad med relativt få tester vilket täcker kravet på

Ett återkommande problem för större integrationstest-sviter som innehåller datalagring i en extern databas är flaky tester. Då implementeringen återanvänder en redan existerande seedingfunktion för att minimera kodduplicering som ännu inte är helt anpassad för testdataseeding resulterade detta i ett flaky testscenario vilket krävde felsökning av utvecklare med större insikt i denna del för att komma tillrätta med problemet.

Sammanfattningsvis så har arbetet resulterat i en grund för framtida utvecklare att stå på för att fortsätta att säkerställa säkerhet och funktionalitet i systemet. Detta har åstadkommits genom ett komplett testflöde av all ny kod som tillkommer i projektet genom implementerad strategi. Dokumentation har utformats för att utvecklare ska förstå och använda testprocessen. På grund av tidsbrist har den nya arbetsmetoden inte ordentligt förankrats hos medarbetare och därför saknas utvärdering av efterlevnad och effektivitet av teststrategin.

6.1 Rekommendationer

Projektets kodbas har genomgått flera stora förändringar under arbetets gång och den undersökande fasen tog längre tid än väntat, vilket inneburit att implementeringen av teststrategin inte skett i tid för en ordentlig utvärdering av hur det fungerar för utvecklare i praktiken. En utvärdering bör ske så snart som möjligt och även löpande för att säkerställa att lösningen verkligen används i praktiken.

Det finns även en önskan om hjälpmedel för att automatisera utvecklingen av tester vilket jag anser vara fullt möjligt med hjälp av de AI-verktyg som redan existerar då utformningen av tester följer en strikt mall och dokumentation för hur tester ska implementeras finns.
I arbetet har fokus enbart legat på backend API:et då det är det mest kritiska. Då projektet även består av en klient i form av en webbsida kommer en ny eller anpassad teststrategi behövas för den delen i framtiden för att säkerställa kvalitet genom hela projektet.

7 Referenser

7.1 Hänvisning till böcker

da Costa, L. (2021). *Testing JavaScript applications.* Manning Publications.

Subramaniam, V. (2016). *Test-Driving JavaScript Applications: Rapid, confident, maintainable code.* The Pragmatic Bookshelf.

Myers, G. J., Sandler, C., & Badgett, T. (2012). *The art of software testing* (3rd ed.). Wiley.

Rasmusson, J. (2017). *The way of the web tester: A Beginner's Guide to Automating Tests.* The Pragmatic Bookshelf.

7.2 Hänvisning till internet/webbsida

Chai. (2026). *Chai assertion library*. Hämtad 2026-05-01, från https://www.chaijs.com

Coe, B. (2018). *Rethinking JavaScript Test Coverage.* https://medium.com/the-node-js-collection/rethinking-javascript-test-coverage-5726fb272949

Engmark, V. (2020). *Start test names with "should".* https://paperless.blog/start-test-names-with-should

Fowler, M. (2012). *Test Coverage.* https://martinfowler.com/bliki/TestCoverage.html

GitHub. (2026). *GitHub docs*. Hämtad 2026-05-03, från https://docs.github.com/en

Jest. (2026). *Delightful JavaScript Testing*. Hämtad 2026-04-26, från https://jestjs.io/

Kan, Y. (20 mars 2024). *Jest vs Mocha: JavaScript Testing Comparison 2026.* https://yrkan.com/blog/jest-vs-mocha-comparison/

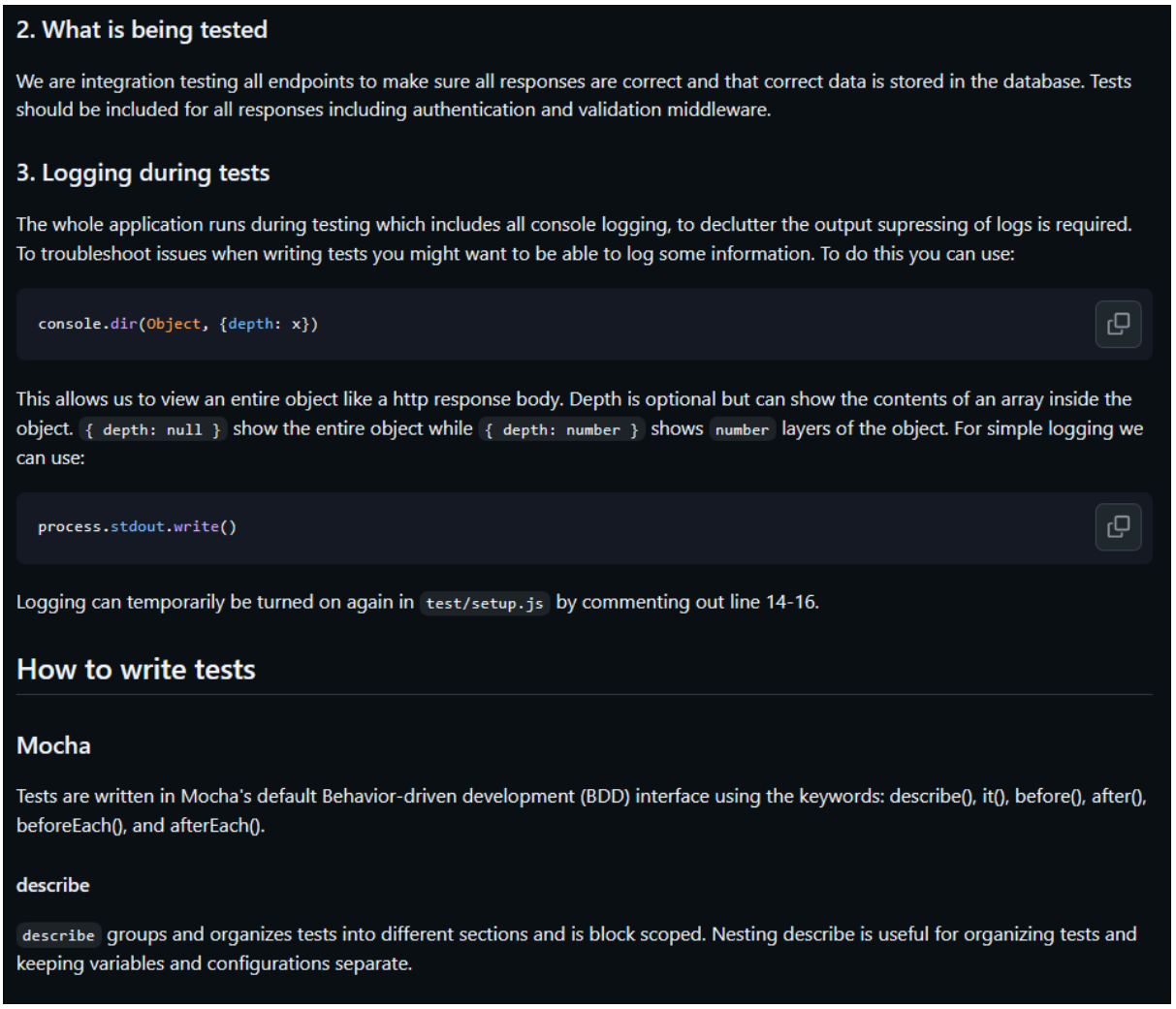
Mocha. (2026). *Mocha | Classic, reliable, trusted.* Hämtad 2026-05-01 från https://mochajs.org/

Mochawesome. (24 februari, 2022). *Mochawesome.* https://github.com/adamgruber/mochawesome

Wagner, T. (24 mars 2025). *Understanding test automation frameworks: What is an Assertion Library?* https://www.trevorwagner.dev/blog/posts/understanding-test-automation-frameworks-what-is-an-assertion-library/

8 Bilagor

Bilaga 1: Utdrag ur testdokumentationen



2. What is being tested

We are integration testing all endpoints to make sure all responses are correct and that correct data is stored in the database. Tests should be included for all responses including authentication and validation middleware.

3. Logging during tests

The whole application runs during testing which includes all console logging, to declutter the output supressing of logs is required. To troubleshoot issues when writing tests you might want to be able to log some information. To do this you can use:

```
console.dir(Object, {depth: x})
```

This allows us to view an entire object like a http response body. Depth is optional but can show the contents of an array inside the object. `{ depth: null }` show the entire object while `{ depth: number }` shows `number` layers of the object. For simple logging we can use:

```
process.stdout.write()
```

Logging can temporarily be turned on again in `test/setup.js` by commenting out line 14-16.

How to write tests

Mocha

Tests are written in Mocha's default Behavior-driven development (BDD) interface using the keywords: describe(), it(), before(), after(), beforeEach(), and afterEach().

describe

`describe` groups and organizes tests into different sections and is block scoped. Nesting describe is useful for organizing tests and keeping variables and configurations separate.

Bilaga 2: Exempel på namngivning

activateTempUserAccount - POST /activate/:loginToken

Validation tests

✔ should return 400 and correct error when token contains invalid characters

✔ should return 500 and correct error when token signature is invalid

Valid token tests

✔ should return 400 and correct error if TempUser is removed before activating

✔ should return 200 and apply all expected response and database changes